job-orchestrator

High-volume job orchestration for computational structural biology

---

Rodrigo V. Honorato, PhD

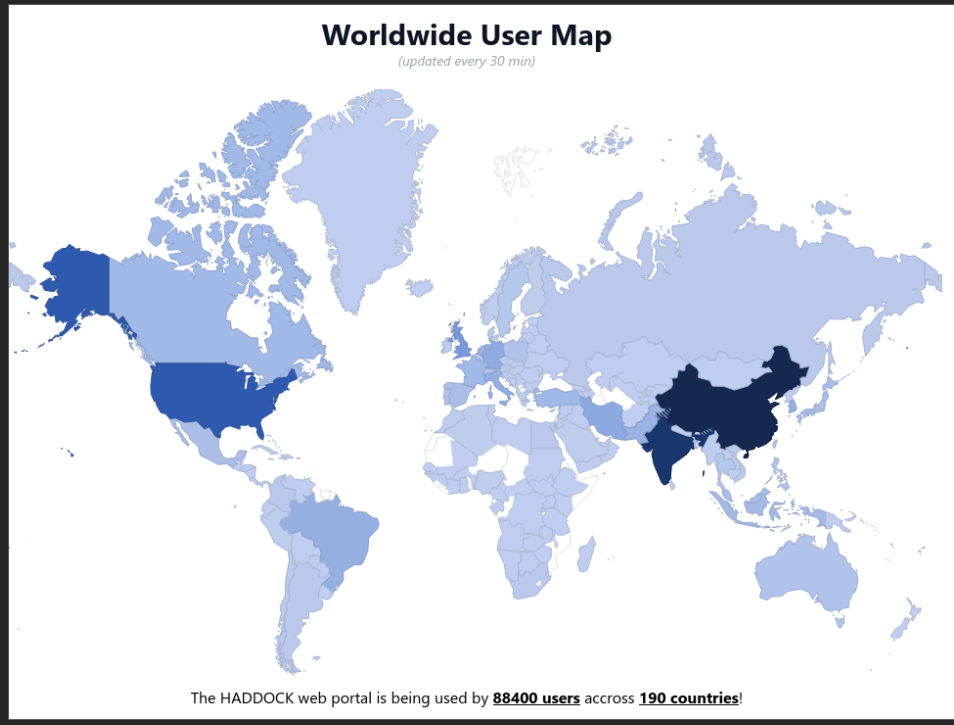
June 2026

Motivation

WeNMR — worldwide e-Infrastructure for NMR & structural biology

90k+	**40k**
registered users	jobs / month

- Portal exposes **research software** as web services
- High submission volume demands **reliable orchestration**
- **Fair allocation** — no single user monopolises slots



Architecture — Single Binary, Dual Mode

Server

- REST API (`/upload`, `/download/{id}`, `/terminate/{id}`)
- Quota enforcement & job routing
- Persistent SQLite (job metadata)
- Background: **sender** + **getter**
- Auto-cleanup after grace period

Client

- Receives job payloads from server
- Executes `run.sh` in isolated dirs
- Ephemeral SQLite (transient state)
- Background: **runner** + **updater**
- Returns results + exit codes

`job-orchestrator server` | `job-orchestrator client` — same binary, one CLI flag

Shared types

Structs defined once, used on both sides — no protocol drift

Tight coupling

Server and client evolve together, in the same commit

Single artefact

One binary to build, version, and ship across all nodes

Tech Stack & Key Choices

evolved from **jobd** → **job-orchestrator** A simpler HTTP-based job dispatch layer — rewritten from scratch for reliability and throughput at scale

Rust

- **Speed** — no garbage collector, predictable latency under load
- **Reliability** — robust and reliable mission-critical component
- **Throughput** — async-native, hundreds of concurrent jobs with low latency

No external services Self-contained binary — nothing to install or provision

SQLite Embedded database — persistent on server, file-based on client (transient)

Async throughout Non-blocking from HTTP layer to storage — no thread-per-job overhead

Fair Scheduling — Round-Robin Quota

- Jobs are grouped by **service** → **user** and dispatched **round-robin**
  - A user sending a large batch cannot fill all available slots
  - Every user gets a slot until the global cap is reached
- Two configurable limits per service:

```
SERVICE_HADDOCK_RUNS_PER_USER=5
SERVICE_HADDOCK_MAX_RUNS=10
```

Server — owns the registry

- Tracks all jobs in persistent SQLite
- Enforces quotas before dispatch
- Queued jobs survive restarts

Client — stateless executor

- No job state of its own
- Sole responsibility: run the payload
- Results and exit codes reported back

Job Lifecycle — 12-State Machine

Jobs move through a well-defined set of states — from submission on the server, through execution on the client, to cleanup. Transitions are driven entirely by async background workers.

State	Description	Transitions to
Queued	Job received and waiting for dispatch	Processing
Processing	Server is sending job to a client	Submitted
Submitted	Job successfully sent to client, awaiting execution	Prepared
Prepared	Payload received by client, ready to execute	Running / Invalid / Failed
Running	Client is actively executing the job	Completed / Failed / Invalid / Killed
Completed	Job finished successfully, results available	Cleaned
Failed	Job failed permanently — non-zero exit code	Cleaned
Invalid	Job rejected — `run.sh` missing, unsafe script, or validation failure	Cleaned
Killed	Job was manually terminated via API	Cleaned
Cleaned	Job data removed after retention period	—
Locked	Temporarily locked during termination or dispatch	—
Unknown	Fallback when status cannot be parsed — retried on next poll cycle	—

Design Detail — File-Based Exit Code IPC (Inter-Process Communication)

The problem

Jobs run as **detached bash processes** — once detached, the exit status is lost. A plain PID watch is not enough: the PID may be reused, and there is no reliable way to capture what the script returned.

The solution

A `trap` in `run.sh` writes the exit code to a file on process exit — regardless of how the process ends. The updater polls for this file every 500 ms. File **presence** means done; file **content** is the exit code.

Required in every `run.sh`:

```
#!/bin/bash
trap 'echo $? > .orchestrator.exit' EXIT

# ... actual computation ...
```

- Works with **kill** — `SIGTERM` triggers the trap, file is written
- PID reuse is safe — file check takes precedence over PID liveness
- No blocking wait — fits naturally into the async poll loop
- Enforced client-side before execution — no trap, no execution

Security

The core risk The client executes an arbitrary bash script submitted by a remote user — this is inherently dangerous and must be treated as untrusted input.

Script validation — before any execution

- UTF-8 validation + 20 MiB size limit — rejects binary payloads
- Regex blocklist: `rm -rf`, `mkfs`, `dd`, sensitive paths (`/etc/passwd`, `/proc/`, `/sys/`, `~/.ssh/`, …)
- Path traversal sanitisation on upload (`../` stripping)

Container hardening — limit blast radius

- Read-only root filesystem · dropped Linux capabilities
- `no-new-privileges` · CPU, memory, and PID limits per container
- Jobs run in isolated directories — no shared state between runs

Note: this is not a sandboxed execution environment. The validation layer is a safeguard, not a security boundary — payloads are expected to come from a **trusted source** (authenticated portal users, not arbitrary internet traffic).

In Production at WeNMR

job-orchestrator is the backbone of the **WeNMR service portal** — a multi-service platform for computational structural biology.

Services orchestrated

- HADDOCK3
- DisVis
- PowerFit
- Arctic3D
- PRODIGY
- Whiscy
- DeepRank
- FANDAS

Deployment

- Docker Compose (current) · Kubernetes manifests ready
- NGINX reverse proxy + TLS termination
- CI via GitHub Actions with integration test suite

What's next The Kubernetes manifests exist — the next step is to battle-test the full deployment under real production load across multiple nodes in a cloud environment.

job-orchestrator

A production Rust job orchestration system for high-volume scientific computing

Architecture	Scheduling	Design	Production
Single binary · dual mode · shared types	Round-robin quota · starvation-free	File-based IPC · 12-state machine	8 services · 90k users · 40k jobs/month

Rodrigo V. Honorato, PhD

rvhonorato@pm.me

github.com/rvhonorato/job-orchestrator

June 2026